Title: SPI Master-Slave Communication with Interrupt-Driven UART Forwarding

Name: KHAN MD SHAKIL
Student ID: 2330130220
Date of Lab: 1st December, 2025

Introduction-

Before this lab, it was understood that SPI (Serial Peripheral Interface) is a synchronous, full-duplex communication protocol commonly used for high-speed data exchange between microcontrollers and peripherals. The lab is about implementing SPI communication between two STM32L431 boards, where one acts as a master transmitting data and the other as a slave receiving data through interrupts and immediately forwarding each byte via UART to a serial terminal. The purpose of this lab was to configure master-slave SPI communication using specific pin connections, implement interrupt-driven data reception on the slave, and verify real-time data forwarding from SPI to UART without polling or delays.

Materials-

Apparatus	Expendables
• Two AHL-STM32L431 development boards	• USB cables
• AHL-GEC-IDE software	• Jumper wires for SPI connections
• PC with serial terminal software	• None

Procedure-

The procedure was followed as outlined in the lab handout. The key steps involved:

1. Connecting the master and slave boards using the specified SPI pins: PTA5↔PTB13 (SCK), PTA6↔PTB14 (MISO), PTA7↔PTB15 (MOSI), and PTA15↔PTB12 (NSS).
2. Configuring SPI_1 as master on one board and SPI_2 as slave on the other board.
3. Enabling receive interrupt on the slave's SPI_2 peripheral.
4. Programming the master to periodically send the string "SPI-Test!\r\n" every 2 seconds.

5. Implementing an interrupt service routine on the slave to read each received byte and forward it immediately via UART.
6. Monitoring the slave's UART output on a serial terminal to verify correct data forwarding.

Data-

Variables Used in the Project:

- send_data[11]: Buffer containing the string "SPI-Test!\r\n" for transmission
- received_byte: Temporary variable storing each byte received in the SPI interrupt

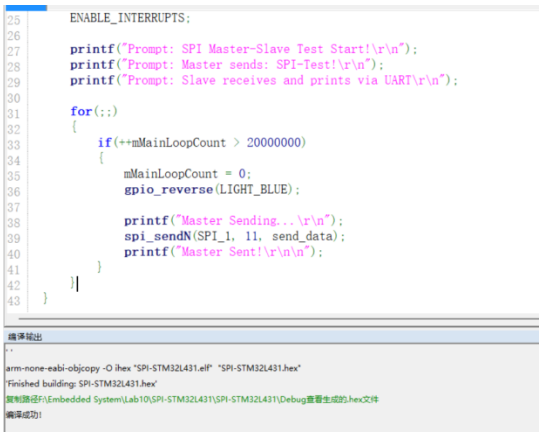
Observed System Behavior:

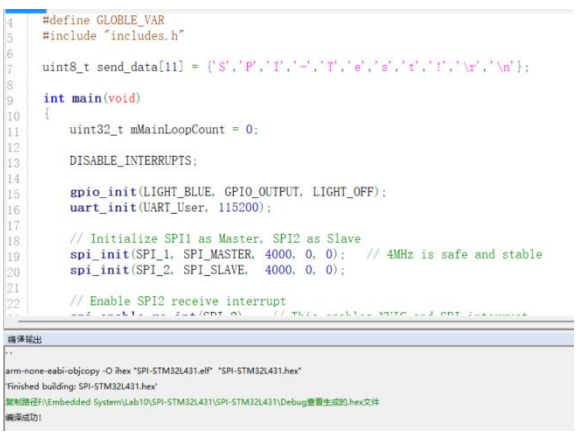
- Master successfully transmitted the 11-byte string every ~2 seconds
- Slave triggered an SPI receive interrupt for each incoming byte
- Each byte was immediately forwarded to the UART_User port
- Serial terminal displayed repeated, clean lines of "SPI-Test!"
- Blue LED on master board blinked once per transmission cycle

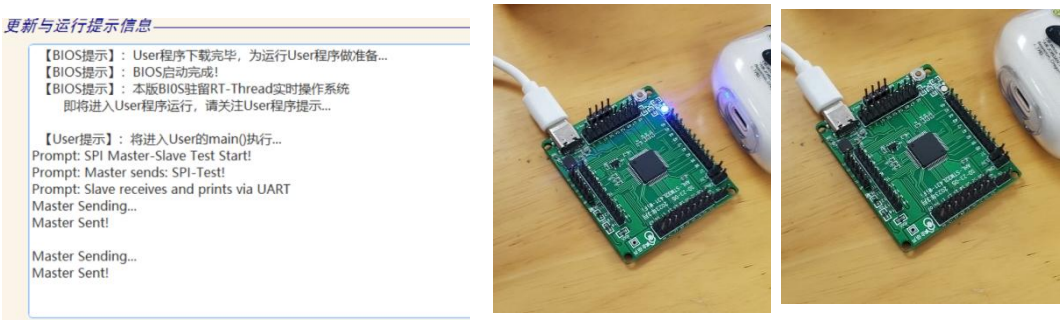
Calculations-

No complex mathematical calculations were required. The timing for periodic transmission was implemented using a software delay loop:

- Delay implemented via a counter of approximately 20,000,000 iterations
- This resulted in a transmission period of ~2 seconds between each data send cycle







Analysis

1. Why was an interrupt-driven approach used instead of polling for SPI reception?
An interrupt-driven approach ensures that the CPU is not continuously busy checking for incoming data. When a byte is received via SPI, the interrupt is triggered immediately, allowing the system to process the data in real-time while freeing the CPU to handle other tasks between receptions. This is more efficient and enables better system responsiveness.

2. How does the specified pin wiring ensure proper SPI communication?
The four-wire SPI connection includes:

- SCK (Serial Clock): Synchronizes data transmission
- MOSI (Master Out Slave In): Data line from master to slave
- MISO (Master In Slave Out): Data line from slave to master (not used in this simplex setup)
- NSS (Slave Select): Enables the slave device
  The specific pin mapping (PTA5↔PTB13, etc.) ensures that both boards use compatible alternate functions for SPI communication.

3. What confirms that data forwarding from SPI to UART is working in real-time?
The clean, uninterrupted display of "SPI-Test!" lines on the serial terminal at the same 2-second interval as the master's transmission confirms real-time operation. If there were delays or data loss, the output would show gaps, corruption, or irregular timing.

Conclusions

The purpose of this lab was to implement SPI master-slave communication with interrupt-driven data reception and real-time UART forwarding. This was successfully accomplished; the master board transmitted data periodically, the slave board received each byte via SPI interrupt, and immediately forwarded it via UART to display correctly on the serial terminal.

The experiment demonstrated precise timing and reliable data transfer. All specified pin connections were correctly implemented and functioned as required. Potential sources of error that

were avoided included incorrect pin mapping, failure to enable SPI receive interrupts, or not clearing the interrupt flag after reading received data.

This project relates directly to real-world applications such as sensor networks where multiple microcontrollers communicate, display systems where data must be processed and forwarded quickly, and industrial control systems requiring reliable, high-speed inter-device communication. The techniques learned are essential for designing efficient, responsive embedded systems that handle multiple peripherals simultaneously.